NATIONAL UNIVERSITY OF SINGAPORE

IT5006 - Fundamentals of Data Analytics

Project Description · AY 2026/27 Semester 1

E-Commerce Analytics

🗓 Thursdays, **6:30–9:30 PM** | 👥 Teams of **3–5** | 📦 **Olist** E-Commerce Dataset | 🎯 3 Phases · **20% / 40% / 40%**

🔍 Introduction to E-Commerce Analytics

E-commerce analytics applies data-driven methods to understand customer behaviour, operational performance, and business outcomes in online retail. Modern platforms generate rich transactional data - orders, payments, deliveries, reviews, and product catalogues - that can be analysed to improve customer experience, reduce operational risk, and support better business decisions.

As online retail continues to grow, organisations rely on analytics to answer questions such as: *Will this order arrive late? Which customers are likely to be dissatisfied? What*

factors drive delivery performance? This project gives you hands-on experience answering questions like these using real e-commerce data and the methods covered in IT5006.

🎯 Types of Analytics Problems (Examples)

E-commerce analytics commonly involves:

1. 📦 **Operational performance** - predicting delivery delays, fulfilment bottlenecks, or order status outcomes
2. ⭐ **Customer experience** - predicting review scores, satisfaction levels, or complaint risk
3. 💰 **Commercial outcomes** - predicting order value, payment behaviour, or product demand

In this project, **your team defines its own analytics problem(s)** within the e-commerce domain, scoped to what you can realistically deliver with the Olist dataset and course techniques.

🔧 Technical Foundation

You will apply supervised learning methods taught in IT5006:

- **Classification** - predicting categorical outcomes (e.g. late vs on-time delivery)
- **Regression** - predicting continuous outcomes (e.g. delivery lead time in days)
- **Model evaluation** - disciplined train/test splits, cross-validation, and comparison against baselines
- **Model selection** - choosing the best approach based on evidence, not model count
- **Ensemble methods** - combining models (e.g. soft voting, stacking) where justified

The emphasis is on **quality of analysis**, not quantity of techniques. You will reuse a **small, deliberate set of model families** across your problem(s) rather than trying many algorithms shallowly (see *Model Family Budget* under Phase 2).

📋 Project Overview

In this project, your team will explore the **Olist Brazilian E-Commerce** dataset, define one or two analytics problems relevant to an e-commerce stakeholder, build and evaluate predictive models, and deploy a proof-of-concept application. The project culminates in a professional report and live presentation.

Learning Objectives

By completing this project, students will:

- Apply end-to-end data analytics methodology to real-world e-commerce data
- Scope and justify analytics problems appropriate to available data and team capacity
- Implement **both classification and regression**, using a small, reusable set of model families (2–3 total)
- Follow a **disciplined train–test paradigm** with proper model selection and evaluation
- Recognise and avoid **data leakage** and **class imbalance pitfalls** specific to real transactional data
- Develop interactive dashboards for stakeholder communication
- Deploy machine learning models as proof-of-concept applications
- Generate actionable insights for an identified e-commerce stakeholder
- Practice professional reporting and presentation skills

👥 Team Formation & Registration

Requirement	Details
Team Size	3 (min) – 5 (max); no individual projects allowed
Registration Deadline	**Sunday, 23 August 2026 at 23:59** (end of Week 2)

Requirement	Details
Submission	Register your team on Canvas
Late Registration	Not permitted - affects final grade

Team Composition (Guidelines)

Teams should include members with complementary skills in:

- 📊 Data analysis
- 💻 Coding
- 📝 Documentation
- 🎤 Presentation
- 👑 Leadership
- 🚀 Deployment & DevOps

📊 Dataset

Olist Brazilian E-Commerce Dataset

The project uses the **Olist Brazilian E-Commerce Public Dataset** - a real-world dataset widely used for teaching data analytics and machine learning.

- **Domain:** E-commerce / online marketplace (Brazil)
- **Period:** 2016 – 2018 (approx.)
- **Format:** CSV (multiple related tables)
- **Download from: Canvas → Week 1 module.** Please use this exact copy rather than downloading it yourself from Kaggle or elsewhere - this ensures every team works with **identical data**, so results are comparable across teams. (The dataset originates from the Kaggle Brazilian E-Commerce Public Dataset by Olist, credited under References below.)

Dataset Tables

Table	Description
`olist_orders_dataset.csv`	Order info, status, purchase/delivery timestamps
`olist_order_items_dataset.csv`	Items per order (product, seller, price, freight)
`olist_order_payments_dataset.csv`	Payment method, instalments, value
`olist_order_reviews_dataset.csv`	Customer review score (1–5) and comments
`olist_customers_dataset.csv`	Customer location (city, state, zip prefix)
`olist_sellers_dataset.csv`	Seller location
`olist_products_dataset.csv`	Product category, dimensions, weight
`olist_geolocation_dataset.csv`	Zip-code geolocation reference
`product_category_name_translation.csv`	Portuguese → English category names

Teams must **join and merge** the relevant tables for their chosen problem(s). Document your data model clearly in all reports.

> 🔑 `customer_id` **vs.** `customer_unique_id` **- read before any customer-level analysis**
>
> In `olist_customers_dataset.csv` these are **not the same thing**:
>
> - `customer_id` - generated **fresh for every order**. It is really an *order-customer* key, not a person. Joining on this makes every order look like a different, first-time customer.
> - `customer_unique_id` - the actual identifier for a real customer, consistent across **all of their orders**.
>
> If your problem involves anything customer-level - repeat-purchase behaviour, customer-level aggregates/features, or making sure the same customer doesn't appear in both your train and test sets - you must group/join on

`customer_unique_id`. Using `customer_id` for this will silently produce wrong results (e.g. "no repeat customers found"). Check which key your problem needs before you start feature engineering.

> ⚠️ **Data Leakage Warning - read before defining your target variable**
> Several `orders` columns are only known **after** an order outcome has already occurred, and can accidentally leak your target if used as predictors:
> - `order_delivered_carrier_date`, `order_delivered_customer_date` - must **not** be used as predictors if your target is "late delivery" or "delivery lead time", since these dates *define* the outcome.
> - `order_approved_at` - usually safe, but check it doesn't post-date information you're trying to predict.
> - Review dates (`review_creation_date`, `review_answer_timestamp`) - must **not** be used to predict review *score*, since the review is written after the order experience.
> Only use features that would realistically be **known at prediction time** (e.g. at the moment an order is placed, or at the moment it ships).

> ⚠️ **Class Imbalance Note**
> Real Olist data is imbalanced for common targets - e.g. most orders are **not** late, and most reviews are **4–5 stars**. Relying on accuracy alone can be misleading (a model that always predicts "on time" may look 90%+ accurate but is useless). Use **Precision/Recall/F1, PR-AUC, or balanced accuracy**, and consider class weights or resampling where appropriate. Discuss this explicitly in your report.

💡 Example Problem Ideas (Your Team Chooses)

These are **starting points only**. Your team must define, justify, and scope its own problem(s). You may adapt one of these or propose something different that the Olist data supports.

Example 1 - Classification: Late Delivery Prediction

- **Question:** Will an order be delivered later than the estimated delivery date?

- **Target:** Binary (late vs on-time)
- **Stakeholder:** Operations / logistics team
- **Features:** Order timing, freight value, seller/customer state, product category, payment type, etc.

Example 2 - Regression: Delivery Lead Time Prediction

- **Question:** How many days from purchase to customer delivery?
- **Target:** Continuous (delivery lead time in days)
- **Stakeholder:** Customer service / fulfilment planning
- **Features:** Distance proxies (customer/seller state), product weight, freight, seasonality, etc.

Example 3 - Classification: Low Customer Satisfaction Risk

- **Question:** Will an order receive a low review score (e.g. 1–2 stars)?
- **Target:** Binary (low satisfaction vs satisfactory)
- **Stakeholder:** Customer experience / seller quality team
- **Features:** Delivery performance, product category, price, payment method, review timing, etc.

Example 4 - Dual Framing (one problem, both model types)

- **Question:** How can delivery performance be predicted *and* acted on?
- **Framing A (regression):** Predict delivery lead time in days
- **Framing B (classification):** Using the same lead-time target, classify orders as late vs on-time against the estimated delivery date
- **Why it works:** One underlying problem, two complementary outputs - satisfies the classification **and** regression requirement without needing a second, unrelated problem
- **Stakeholder:** Operations team - wants both a precise delay estimate (regression) and a simple flag for at-risk orders (classification)

> **Team scoping options:**
> - **Two problems** - propose one classification and one regression problem (e.g.

> Examples 1 + 2), **or**
> - **One problem, dual framing** - propose a single problem framed as both a classification and a regression task (see Example 4)
>
> Either option satisfies the "both classification and regression" requirement. Discuss your scope with your instructor if unsure.

✅ Problem Scoping Checklist

Before finalising your problem(s) in Phase 2, confirm:

1. **Derivable target** - can your target variable be computed directly from the provided tables (no external data needed)?
2. **Realistic features** - are your predictors all known *before* the outcome occurs (no leakage - see warning above)?
3. **Sufficient signal** - does exploratory analysis (Phase 1) show at least some relationship between candidate features and the target?
4. **Manageable imbalance** - if your target is imbalanced, do you have a plan for appropriate metrics/handling?
5. **Clear stakeholder** - can you name a real business role who would use this prediction, and how?

🗓️ Project Timeline & Deliverables

Instructional period: Weeks 1–13, spanning 10 August – 13 November 2026, with a one-week recess in between.

Recess: 19 – 27 September 2026 (no classes)

Class day: Thursday evenings, 6:30 – 9:30 PM

Note: All submission dates below are also listed on Canvas. For any changes, refer to Canvas.

Phase / Milestone 1: Foundation - Literature Review & Exploratory Data Analysis

Deadline: Sunday, 13 September 2026 at 23:59 (end of Week 5)
Weight: 20%
Report length: 4–5 pages (excludes cover page, references, appendices)

Deliverables

Literature Review (approx. 2 pages)
- Summary of e-commerce analytics approaches relevant to your exploration
- Review of modelling and evaluation methods (classification and regression)
- Feature engineering techniques applicable to transactional data
- Evaluation metrics and their appropriateness (including for imbalanced targets)
- Conclusion: key trends, gaps, and opportunities

Exploratory Data Analysis (approx. 2–3 pages)
- Dataset overview: tables used, joins, row counts, missing values
- Temporal patterns (order volume, seasonality)
- Customer / seller / product distribution analysis
- Correlation and relationship exploration
- **Candidate problem exploration** - preliminary investigation of 2–3 possible analytics problems to inform your Phase 2 scoping (you do not need to finalise the problem yet); use the **Problem Scoping Checklist** above as a guide
- Key insights and patterns discovered

Interactive Dashboard
- Built using **Streamlit** (recommended), Tableau Public, or Power BI
- Interactive visualisations of e-commerce patterns (orders, delivery, reviews, categories, geography, etc.)
- Submit as a live link (include in your report)

Submission Format

- Combined PDF report (Literature Review + EDA) with dashboard link and GitHub repository link
- GitHub repository with all raw code / notebooks

Phase / Milestone 2: Analytics Implementation - Problem Definition, Model Building & Evaluation

Deadline: Sunday, 11 October 2026 at 23:59 (end of Week 8)
Weight: 40%
Report length: 6–8 pages (excludes cover page, references, appendices)

Problem Scoping (Required)

Each team must define **1 or 2 analytics problems maximum**, submitted as part of this phase:

Requirement	Details
Number of problems	**1 or 2** (not more)
Problem types	Must cover **both classification and regression** - via two problems (one of each), **or** one problem with dual framing (see Example 4)
Scope	Must be achievable with Olist data and course techniques
Justification	Business stakeholder, target variable definition, and success criteria

Use the **recess period (19–27 Sep)** to discuss and finalise your problem scope as a team.

Modelling Requirements - Quality Over Quantity

Focus on doing a **small number of model families well**, not trying many algorithms superficially. Many scikit-learn algorithms come in matched Classifier/Regressor pairs - reuse the **same family** across your classification and regression task(s) wherever sensible.

Model Family Budget (2–3 families total for the whole project). The families and algorithms below are **illustrative examples, not a fixed list** - pick any algorithm(s) that reasonably belong to a family you choose, as long as you stay within the 2–3 family budget.

Family (example)	Example classification model(s)	Example regression model(s)
A - Linear	e.g. `LogisticRegression`	e.g. `LinearRegression` / `Ridge`
B - Tree-based	e.g. `DecisionTreeClassifier` / `RandomForestClassifier`	e.g. `DecisionTreeRegressor` / `RandomForestRegressor`
C - Ensemble *(optional, advanced)*	e.g. `VotingClassifier` / `StackingClassifier` (built from A + B)	e.g. `VotingRegressor` / `StackingRegressor` (built from A + B)

> **Baseline tip:** Don't jump straight to your most complex model. Within each family, start with its **simplest variant** as your baseline before adding complexity - e.g. plain `LogisticRegression` / `LinearRegression` before `Ridge` /regularised versions, or a single `DecisionTree` before `RandomForest`. This gives you an honest before/after comparison and lets you show *why* the added complexity was worth it.

Guideline	Details
Model families	**2–3 total** across the entire project (e.g. Linear + Tree-based, optionally + Ensemble - see examples above, other families are welcome)
Baseline	Use the **simplest variant within each family** as your starting point (see tip above); every more complex model must be shown to beat it
Train–test discipline	Clear train/test split (or train/validation/test); **no data leakage** (see warning above)
Model selection	Compare families systematically using appropriate metrics; explain your final choice
Cross-validation	Use CV for robust performance estimates and hyperparameter tuning
Reproducibility	Set random seeds; document all preprocessing steps in pipelines

> **Do not** implement many algorithms without depth. A well-evaluated comparison of 2–3 model families - reused sensibly across classification and regression - with clear business interpretation is worth more than a long list of poorly tuned models.

Deliverables

- Problem statement(s) and stakeholder context (1–2 problems max)
- Data preprocessing and cleaning methodology
- Feature engineering strategy
- Model descriptions and training process (2–3 model families total)
- Hyperparameter tuning methodology
- **Classification metrics:** Precision, Recall, F1-Score, AUC-ROC / PR-AUC (as appropriate for imbalance)
- **Regression metrics:** MAE, RMSE, R^2 (as appropriate)
- Cross-validation results
- Feature importance / interpretability analysis
- Model comparison and final selection with justification
- Ensemble or stacking results (if used)
- Actionable insights for your e-commerce stakeholder
- Discussion of limitations and constraints

Submission Format

- Technical report as PDF with GitHub repository link
- GitHub repository with Jupyter notebooks and Python scripts
- Model performance summary tables (in report)

Phase / Milestone 3: Integration & Communication - Deployment, Final Report & Presentation

Weight: 40%
Report length: 15–20 pages (excludes cover page, references, appendices)

Presentation Slides Deadline (Same Deadline for Every Team)

Wednesday, 4 November 2026 at 23:59 - one single deadline for **all** teams, regardless of whether you are presenting on 5 Nov or 12 Nov.

- Submit your presentation slides (PDF or PPT) via Canvas
- This is the **exact deck** you will present live, and the exact deck included in your final ZIP submission
- **No changes to slides are permitted after this deadline** - this ensures total fairness, since no team should have extra preparation time or the benefit of watching other teams present before finalising their own slides
- Late slide submission follows the same late penalty policy as other deliverables

Final Report & ZIP Deadline

Sunday, 8 November 2026 at 23:59 (end of Week 12)

Submit as ZIP file containing:
- Final report PDF (include application link and GitHub link)
- Presentation slides (identical to the version locked on 4 Nov - see above)
- GitHub repository link

Live Presentations

Thursday, 5 November and Thursday, 12 November 2026 (Weeks 12–13, during class 6:30 – 9:30 PM)

- Presentation order will be communicated by instructors
- **10 minutes** presentation + **5 minutes** Q&A
- All teams present using the slide deck locked on 4 Nov (see above); all teams must be prepared to present on their assigned date

Peer Evaluation Deadline

Sunday, 15 November 2026 at 23:59

Deliverables

Final Report

- Cover page
- Executive summary
- Dataset and preprocessing
- Problem definition and stakeholder context
- Exploratory analysis highlights
- Modelling approaches (classification and regression)
- Results and evaluation
- Model deployment and application
- Business recommendations and impact
- References

Deployed Application (POC / MVP)

- Streamlit Cloud (recommended), FastAPI, Gradio, or similar
- Live URL included in final report
- Demonstrates at least one deployed model from your project
- Clear usage instructions
- Note: since Olist data covers 2016–2018 historical orders, your app predicts on **held-out historical records** simulating new orders - it does not need to connect to a live order feed

GitHub Repository

- Well-documented repository with all project code
- README with setup instructions and project overview
- Organised folder structure:

```
project-root/
├── data/
├── notebooks/
├── src/
├── deployment/
├── docs/
└── README.md
```

- `requirements.txt` or `environment.yml`

- GitHub link included in all report submissions

Presentation Slides
- Professional slides (PowerPoint or PDF)
- Include screenshots of deployed application
- Submit with final report in ZIP file

Peer Evaluation
- Individual contribution assessment
- May be used for individual grade adjustment
- Submitted separately via Canvas survey

Submission Format

- Presentation slides (standalone, locked): due **Wed, 4 Nov** - same deadline for all teams
- ZIP file: final report PDF + presentation slides + GitHub link - due **Sun, 8 Nov**
- Live presentation during assigned Thursday class session (5 or 12 Nov), using the locked slide deck

---

📋 Project Timeline Summary

Milestone	Task	Deliverable	Due Date	Weight
0	👥 Team Formation	Canvas signup	**Sun, 23 Aug 2026**	-
1	📚 Literature Survey & EDA	Report + Dashboard	**Sun, 13 Sep 2026**	20%
-	🛌 Recess	Team problem scoping (informal)	19–27 Sep 2026	-

Milestone	Task	Deliverable	Due Date	Weight
2	💻 Problem Definition & Modelling	Technical Report + Code (1–2 problems, 2–3 model families total)	**Sun, 11 Oct 2026**	40%
3	🔒 Presentation Slides (locked, all teams)	Slide deck (PDF/PPT)	**Wed, 4 Nov 2026**	(in Phase 3)
3	🚀 Deployment & Final Report	Report + Deployment + ZIP	**Sun, 8 Nov 2026**	40%*
3	🎤 Live Presentations	Team presentation (in class)	**Thu, 5 & 12 Nov 2026**	
3	📝 Peer Evaluation	Individual assessment	**Sun, 15 Nov 2026**	

* Deployment & Final Report, Live Presentations, and Peer Evaluation are graded together as **one combined 40% (Phase 3)** - they are not separate or additional weights. Peer Evaluation results are used to make **individual grade adjustments** within that 40%.

Reading week: 14 – 20 November 2026
Examinations: 21 November – 5 December 2026

🎯 Assessment Criteria

Phase 1 (20%)

- Literature review quality and relevance to e-commerce analytics
- EDA depth and thoroughness

- Candidate problem exploration
- Dashboard functionality and clarity
- Report clarity

Phase 2 (40%)

- Problem framing and scoping (appropriate, justified, achievable)
- Data preparation and feature engineering (including leakage avoidance)
- Modelling discipline (train–test, no leakage, reproducibility)
- Quality of model family comparison (not breadth of techniques)
- Evaluation rigor (appropriate metrics for classification, regression, and imbalance)
- Ensemble / stacking (if used) - justified and well explained
- Business interpretation for e-commerce stakeholder

Phase 3 (40%)

- Report integration and coherence
- Model deployment (POC / MVP)
- Presentation quality
- Business impact and recommendations
- Technical excellence

🛠 Technical Requirements

Required Tools

- **Python:** pandas, scikit-learn, matplotlib/seaborn, numpy
- **Dashboard:** Streamlit (recommended), Tableau Public, or Power BI
- **Deployment:** Streamlit Cloud, FastAPI, Gradio, or similar
- **Version Control:** GitHub repository (mandatory)
- **Documentation:** Jupyter Notebooks with markdown explanations

Modelling Guidelines

- Use **scikit-learn Pipelines** where possible to prevent data leakage
- Start with the **simplest variant in your chosen family** as a baseline, then justify any added complexity with evidence (e.g. plain `LinearRegression` / `LogisticRegression` before `Ridge` / `RandomForest`)
- Limit to **2–3 model families total**; reuse families across classification and regression tasks (see Model Family Budget)
- Use **stratified splits** for classification with imbalanced targets
- Set and document **random seeds** for reproducibility
- Ensemble methods (voting, stacking) are encouraged when they add demonstrable value, built from your chosen families
- Watch for **target leakage** via post-outcome columns (see Data Leakage Warning)
- Watch for **class imbalance**; do not rely on accuracy alone (see Class Imbalance Note)

Coding Guidelines

- Clean, well-commented code with meaningful variable names
- Reproducible analysis with random seeds set
- Error handling and data validation checks
- Clear separation of data preparation, modelling, evaluation, and deployment
- All code in GitHub with descriptive commit messages
- Comprehensive README with setup and execution instructions

Data Usage Guidelines

- Use the **Olist dataset** as the primary (and only required) data source
- External data is **not required**; if used, must be justified and documented
- Handle missing values and outliers appropriately
- Document all data transformations and assumptions
- Ensure reproducibility of results

Deployment Guidelines

- Application must be accessible via live URL
- Include error handling for edge cases
- Provide clear usage instructions
- Document any API endpoints with examples
- No hardcoded credentials
- Test deployment before submission

📚 Support Resources

Academic Support

- **Consultation:** Available on request (recommended once every 2 weeks)
- **Canvas Discussion Forum:** Technical questions and peer assistance
- **Email Support:** Course instructors available for guidance
- **Class time:** Thursdays 6:30 – 9:30 PM - use for progress check-ins and Q&A

⚠️ Important Policies

Academic Integrity

- Cite all sources and reference materials appropriately
- Original analysis and interpretation required
- Collaboration within teams encouraged; between teams discouraged
- Use of AI tools must be declared and appropriately credited

Late Submission Policy (All Phases)

- 0–24 hours late: 10% penalty
- 24–48 hours late: 20% penalty
- More than 48 hours late: 50% penalty
- More than 7 days late: Zero marks

Formatting Requirements

- All reports in PDF format, 12pt font, single-spaced
- Figure captions and table labels required
- Professional formatting with consistent styling
- File naming convention:

```
TeamX_PhaseY_IT5006_AY2627Sem1.pdf
TeamX_IT5006_Ecommerce_Analytics_AY2627Sem1.zip
```

- GitHub repository naming:

```
TeamX_IT5006_Ecommerce_Analytics_AY2627Sem1
```

💡 Useful Pointers

1. **Start early** - begin exploring Olist data immediately after team formation
2. **Use recess wisely** - finalise your 1–2 problem(s) and plan Phase 2 during 19–27 Sep
3. **Scope carefully** - one well-executed problem beats two rushed ones; two is the maximum, not the target
4. **Fewer model families, better evaluation** - 2–3 well-tuned, reused model families beat many shallow experiments
5. **Check for leakage first** - before modelling, list which columns are known *before* your prediction point
6. **Pipelines prevent leakage** - fit preprocessing inside cross-validation folds

7. **Mind the customer ID gotcha** - use `customer_unique_id`, not `customer_id`, for any repeat-customer or customer-level logic (see Dataset section)
8. **Regular meetings** - weekly team check-ins with documented minutes
9. **Version control** - commit frequently with descriptive messages
10. **Focus on business value** - connect findings to an e-commerce stakeholder (operations, CX, seller management)
11. **Test deployment early** - a simple working POC beats a complex broken app
12. **Plan for demo** - prepare specific examples for your live presentation

---

📇 Contact Information

Prakash Sukhwal
Senior Lecturer, DISA SOC
Email: prakash.sukhwal@nus.edu.sg

📚 References & Suggested Readings

1. Olist Brazilian E-Commerce Dataset - Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce *(cited for provenance; download your copy from Canvas → Week 1, not from Kaggle - see Dataset section)*
2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.
3. Müller, A. C., & Guido, S. (2021). *Introduction to Machine Learning with Python*. O'Reilly Media.
4. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
5. Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241-259.
6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, 785-794.

7. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.
8. scikit-learn Developers. Ensemble methods (Voting, Stacking) - scikit-learn User Guide. https://scikit-learn.org/stable/modules/ensemble.html

---

Good luck with your analytics journey! We look forward to seeing your innovative insights and professional growth.

For questions or clarifications, please contact the course instructors via email or schedule consultation sessions as needed.

Students are encouraged to start exploring the Olist dataset and discussing problem scope with their team as early as Week 1.

This page can be saved as PDF (Ctrl/Cmd + P) for offline reference. For the latest deadlines, always check Canvas.